

AdaptiveTile: Content-Aware Parallel Image Processing with Predictive Tile Scheduling

EDA & Prototype Submission Report

Author: Humaira Ayesha

Student ID: 23301683

Course: PARALLEL, DISTRIBUTED, AND HIGH-PERFORMANCE
COMPUTING

Institution: Brac University, Department of Computer Science and
Engineering

Date: 5 May 2026

Contents

1	Project Overview	2
1.1	Research Objectives	2
1.2	System Architecture	2
1.3	Datasets	2
2	Exploratory Data Analysis (EDA)	3
2.1	Tile Profiling Procedure	3
2.2	Target Variable: Per-Tile Runtime	3
2.3	Feature Distributions	3
2.4	Outlier Detection	3
2.5	Feature–Runtime Correlation	4
2.6	Train/Test Distributional Shift	5
2.7	Dimensionality Reduction	5
2.8	Preprocessing Pipeline Summary	5
3	Prototype Implementation	6
3.1	Module Structure	6
3.2	Configuration	7
3.3	Tiling with Halo Borders	8
3.4	Feature Extraction	9
3.5	Complexity Predictor	10
3.6	Scheduler Variants	11
3.7	CPU Processing Pipelines	12
4	Predictor Evaluation (Prototype Results)	12
4.1	Model Comparison	13
5	Scheduling & Parallelism Prototype	13
6	Limitations and Future Directions	16
6.1	Current Limitations	16
6.2	Future Work	16
	Conclusion	16

1 Project Overview

AdaptiveTile is an end-to-end system architecture of content-aware parallel image processing. The basic concept comes to the fact that image tiles are not computationally uniform. Rich textures, sharp edges, or complex gradients in tiles demand much greater processing time than smooth areas. The common allocation of work does not recognize this heterogeneity leading to imbalance in loads, and impoverished parallel efficiency.

1.1 Research Objectives

1. **Feature Engineering:** Retrieve lightweight per-tile content characteristics that are associated with processing time.
2. **Complexity Prediction:** Train machine learning regressors to predict per-tile processing time using content features.
3. **Adaptive Scheduling:** Compare four variants of schedulers — Static, Dynamic, Predictive, and Hybrid — on benchmark pictures.
4. **CPU-GPU Evaluation:** Measure speedup, parallel efficiency, and output quality in four execution modes.
5. **Generalisation:** Test scores on three publicly available datasets: Kodak, DIV2K and BSDS500.

1.2 System Architecture

Figure 1 shows the high-level pipeline of AdaptiveTile.

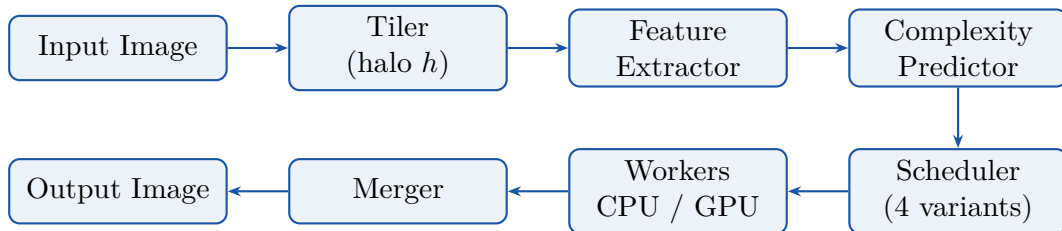


Figure 1: AdaptiveTile system architecture: tiling $\rightarrow$ feature extraction $\rightarrow$ complexity prediction $\rightarrow$ scheduling $\rightarrow$ parallel execution $\rightarrow$ merging.

1.3 Datasets

Three publicly available benchmark datasets are used:

Table 1: Dataset summary.

Dataset	Resolution	Images	Format	Role
Kodak	768×512 / 512×768	24	PNG	Primary profiling & benchmarking
DIV2K	$\approx 2048 \times 1080$	900	PNG	Generalisation (high-res)
BSDS500	481×321	500	JPG	Generalisation (low-res)

Default experimental parameters: tile size $s = 128$ px, halo $h = 16$ px, workers $W \in \{1, 2, 4, 6, 8\}$, GPU streams $S = 4$.

2 Exploratory Data Analysis (EDA)

2.1 Tile Profiling Procedure

The tile size of 128px and halo 16px tile it generates the 24 Kodak images. The pictures consist of 24 tiles and therefore, there are 576 tiles. All the tiles are processed. three times in Pipeline B (denoise threshold) the median To offset system jitter, the runtime is measured. This provides a data set of 576. The number of feature observations, spatial index observations, runtime target observation (ms).

2.2 Target Variable: Per-Tile Runtime

Table 2: Descriptive statistics of per-tile runtime (ms), 576 raw tiles.

Statistic	Value
Minimum	0.159 ms
Maximum	0.668 ms
Mean	0.317 ms
Std Dev	0.084 ms
CV	26.55%
Skewness	0.89
Kurtosis	0.97
Normality	Non-normal ($p < 0.001$, Shapiro-Wilk)

The coefficient of variation of 26.55% confirms meaningful heterogeneity in tile complexity, sufficient to motivate predictive scheduling.

2.3 Feature Distributions

Seven content features are extracted per tile.

Table 3: Summary statistics for all features (576 raw tiles).

Feature	Mean	Std	Skew	Kurt	Normal?
Edge density	0.147	0.098	0.23	-0.87	No
Gradient variance	7211.6	6779.6	1.73	2.87	No
Intensity variance	1523.9	1473.9	2.33	5.41	No
Histogram entropy	1.825	0.492	-1.00	0.08	No
LBP texture score	5.184	0.354	2.41	6.29	No

2.4 Outlier Detection

A majority-vote ensemble (IQR, MAD, 3σ) flags tiles as outliers if at least 2 of 3 rules agree. Result: **64 tiles removed (11.1%)**, leaving 512 clean tiles.

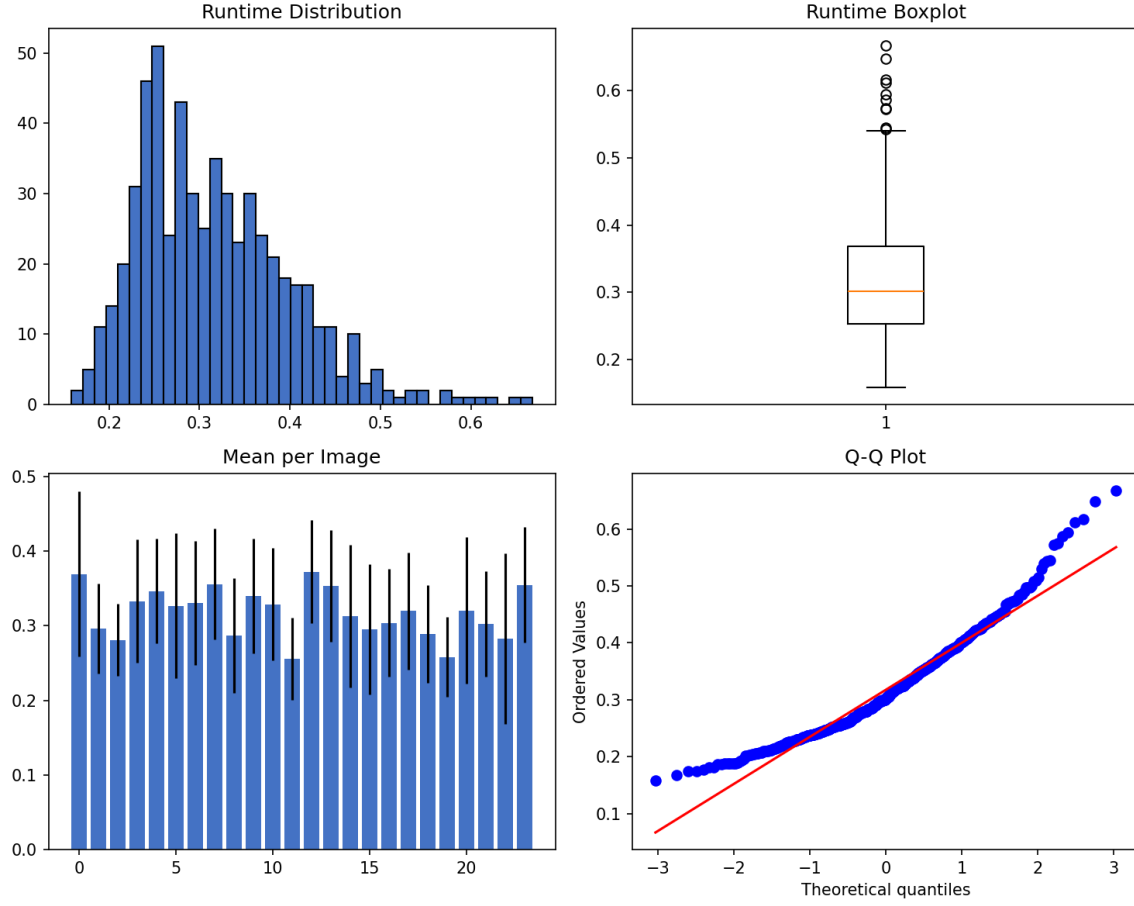


Figure 2: Per-tile runtime distribution. Right-skewed histogram and Q-Q plot confirming non-normality.

2.5 Feature–Runtime Correlation

Table 4: Pearson and Spearman correlation of features with runtime ($p < 0.001$).

Feature	Pearson r	Spearman ρ
Histogram entropy	+0.445	+0.471
Edge density	+0.439	+0.488
Gradient variance	+0.400	+0.493
Intensity variance	+0.197	+0.357
LBP texture score	-0.359	-0.411
Tile row	+0.074	+0.119
Tile col	-0.078	-0.053

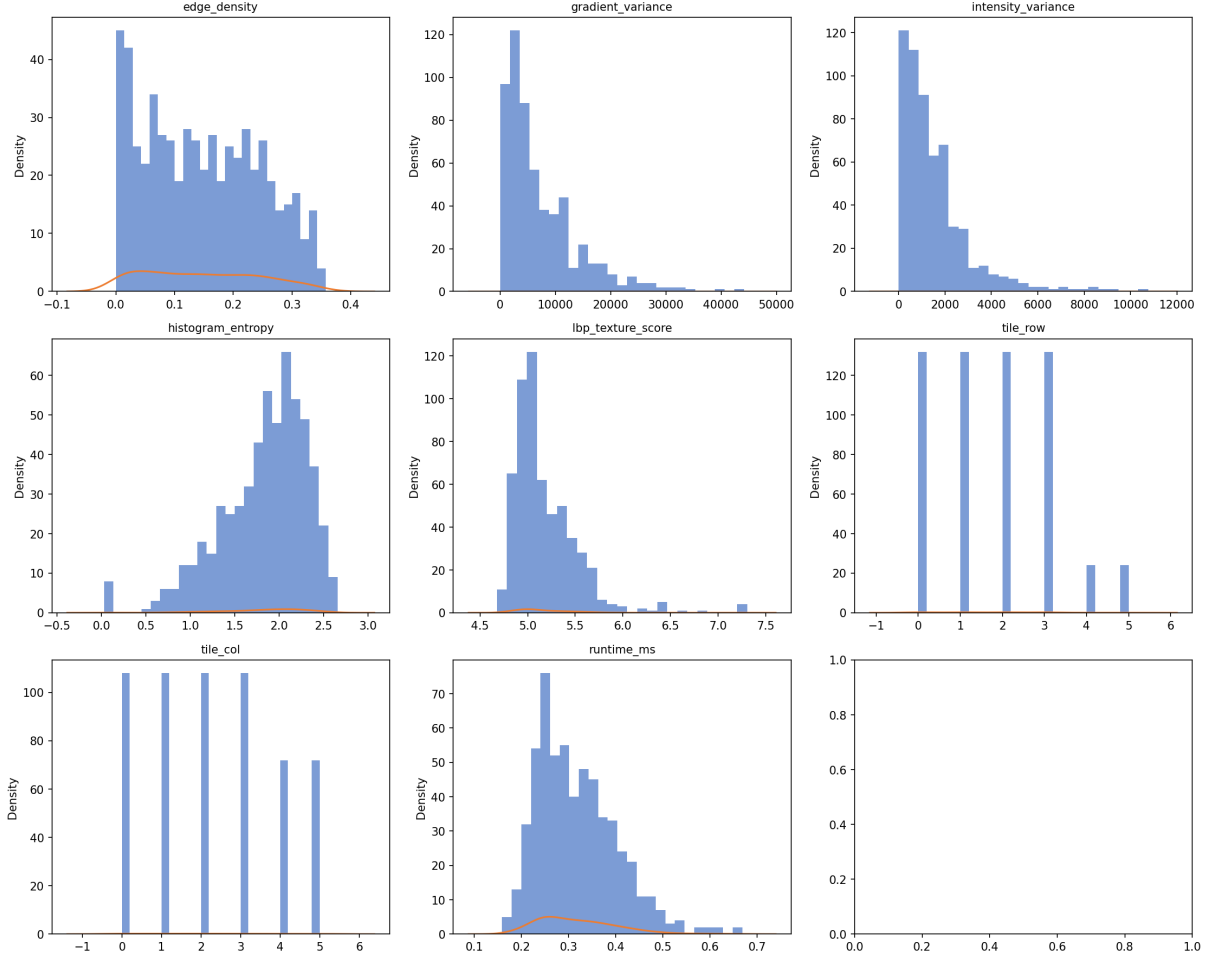


Figure 3: Feature distributions for 576 raw tiles (3x3 grid).

2.6 Train/Test Distributional Shift

Table 5: Train/test distributional shift (PSI > 0.2 = significant).

Feature	PSI	JSD	Drift?
Edge density	0.133	0.044	No
Gradient variance	0.052	0.026	No
Intensity variance	0.170	0.034	No
Histogram entropy	0.229	0.061	Yes
LBP texture score	0.223	0.034	Yes
Runtime (ms)	0.254	0.054	Yes

2.7 Dimensionality Reduction

2.8 Preprocessing Pipeline Summary

Three-step preprocessing based on EDA findings: (1) Majority-vote outlier removal (64 tiles, 11.1%); (2) Yeo–Johnson power transformation for skewed features; (3) RobustScaler normalisation.

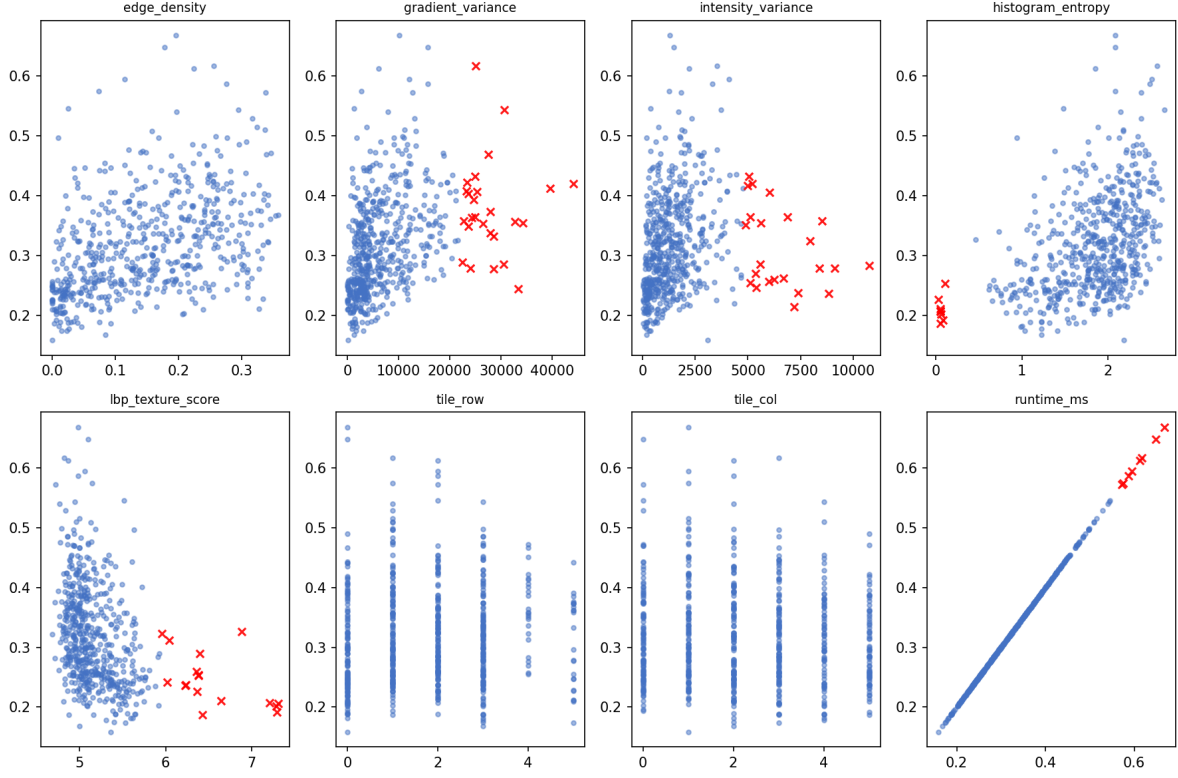


Figure 4: Outlier detection across all features. Flagged tiles shown in red.

3 Prototype Implementation

The prototype implements the full AdaptiveTile pipeline end-to-end in Python, covering tiling, feature extraction, complexity prediction, scheduling, parallel CPU execution, and tile merging.

3.1 Module Structure

Table 6: Source module overview.

Module	Responsibility
src/config.py	Central configuration (tile size, halo, pipeline params)
src/tiling.py	Extract tiles with halo borders from an image
src/features.py	Compute 7 content features per tile
src/predictor.py	Train and evaluate tabular + CNN complexity predictors
src/scheduler.py	Static, Dynamic, Predictive, Hybrid schedulers
src/worker_cpu.py	CPU processing pipelines A and B
src/worker_gpu.py	GPU (Kornia) processing pipeline
src/merge.py	Stitch processed tiles back into full image
src/metrics.py	Speedup, efficiency, imbalance ratio, PSNR, SSIM

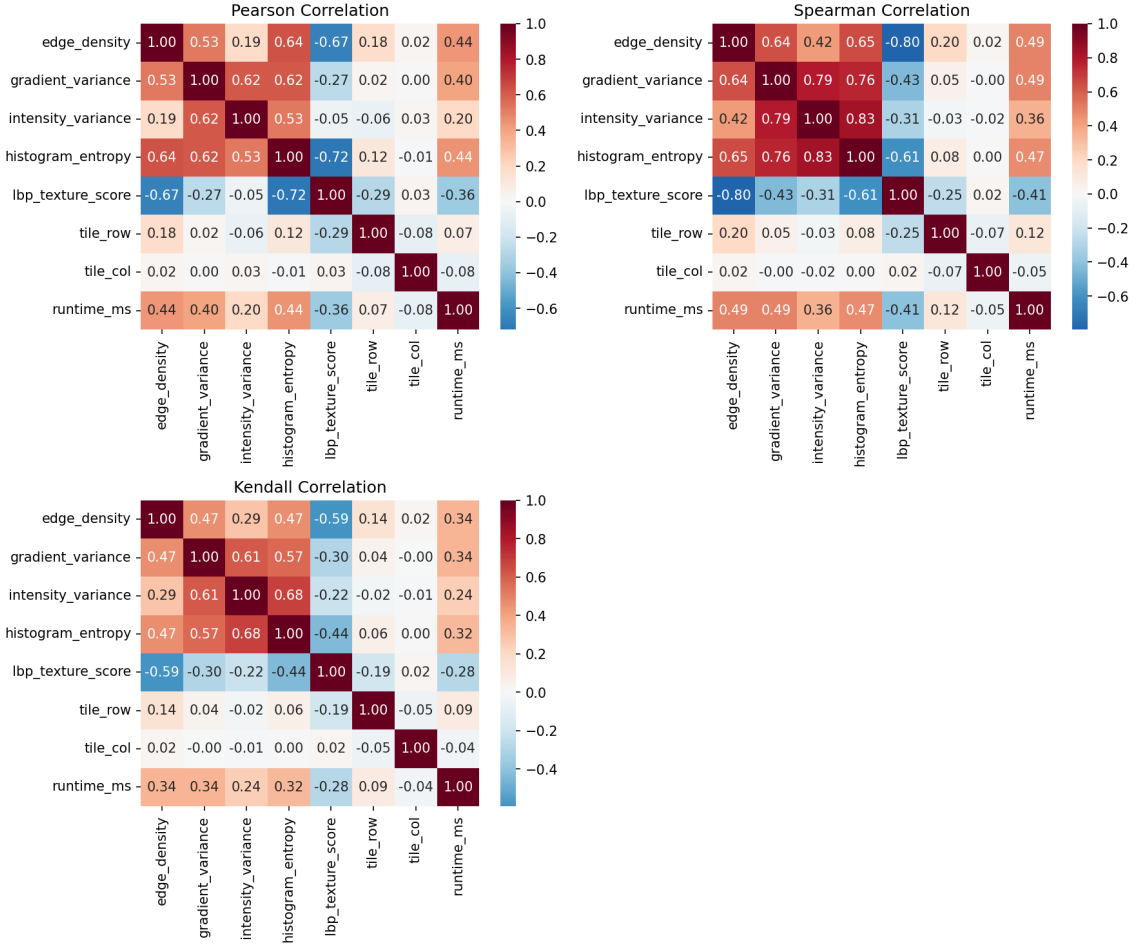


Figure 5: Pearson, Spearman, and Kendall correlation heatmaps.

3.2 Configuration

All hyperparameters are centralised in `src/config.py`, enabling reproducible experiments:

```

1 TILE_SIZE           = 128                # default tile side length
   (pixels)
2 HALO                = 16                # halo border width (pixels)
3 NUM_WORKERS         = min(8, cpu_count())
4 NUM_GPU_STREAMS     = 4
5
6 CANNY_LOW, CANNY_HIGH = 50, 150
7 BILATERAL_D, BILATERAL_SIGMA = 9, 75
8 NLM_H               = 10
9 MORPH_KERNEL_A, MORPH_KERNEL_B = 5, 3
10
11 CNN_EPOCHS, CNN_BATCH_SIZE = 30, 32
12 CNN_LR, CNN_IMG_SIZE = 1e-4, 224
13
14 TILE_SIZES          = [128, 256, 512, 1024]
15 HALOS                = [4, 8, 16, 32]
16 WORKER_COUNTS       = [1, 2, 4, 8]

```

Listing 1: Central configuration (`src/config.py`).

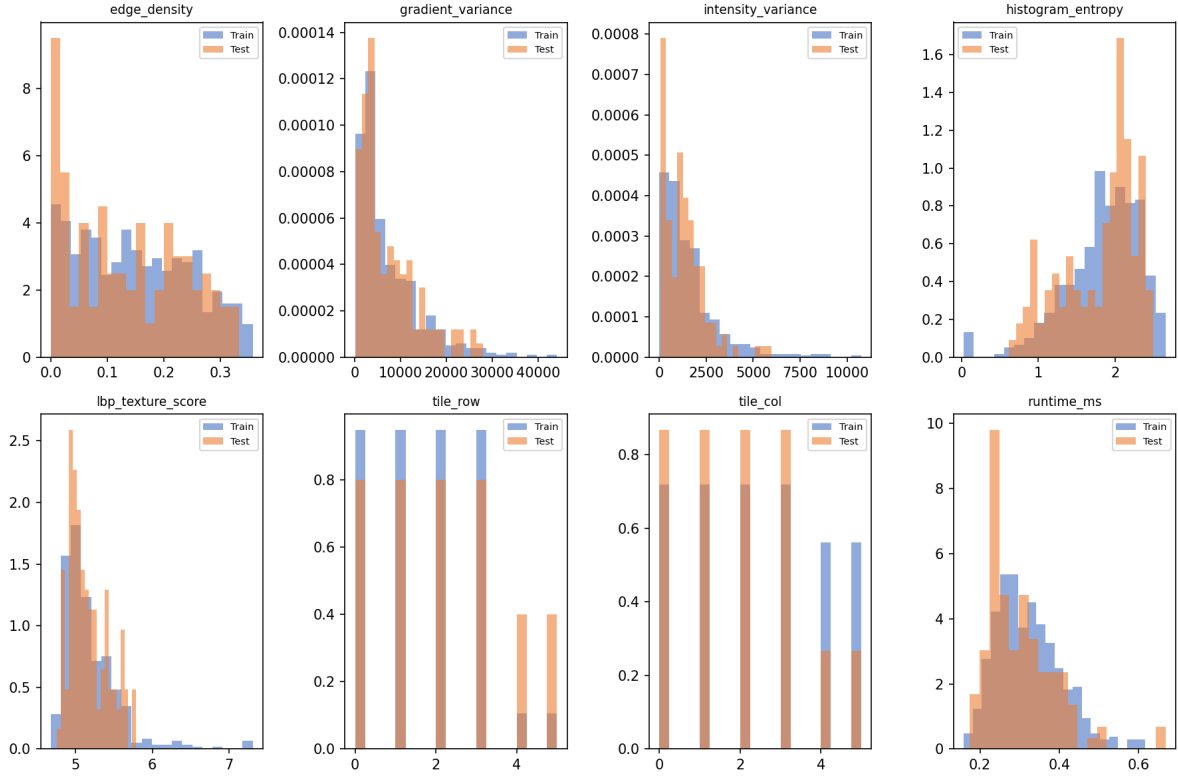


Figure 6: Train vs. test feature distributions (GroupShuffleSplit 80/20).

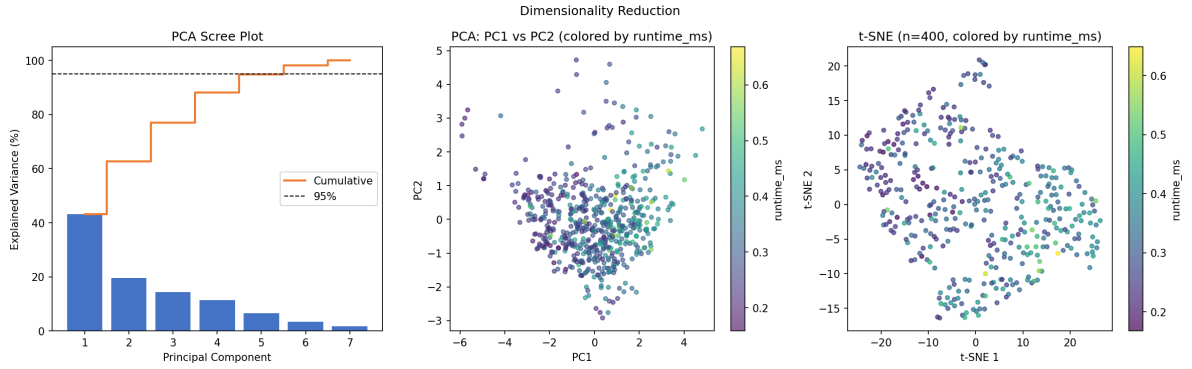


Figure 7: Left: PCA (6 components = 95% variance). Right: t-SNE showing three runtime clusters (fast, medium, slow tiles).

3.3 Tiling with Halo Borders

The tiler decomposes an image into $n_r \times n_c$ tiles each padded with halo h pixels (clamped to image boundaries):

```

1 @dataclass
2 class TileInfo:
3     tile_id: int
4     row: int; col: int
5     x_start: int; y_start: int; x_end: int; y_end: int
6     left_halo: int; top_halo: int
7     right_halo: int; bottom_halo: int
8     data: np.ndarray    # halo-padded pixel array

```

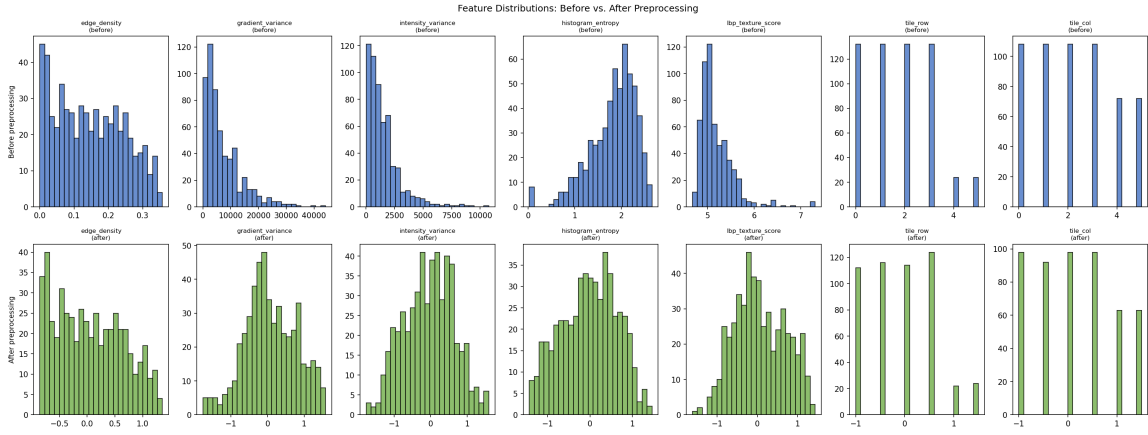


Figure 8: Feature distributions before and after preprocessing.

```

9
10 def extract_tiles(image: np.ndarray,
11                  tile_size: int, halo: int) -> list[TileInfo]:
12     h, w = image.shape[:2]
13     tiles, tid = [], 0
14     for r in range(math.ceil(h / tile_size)):
15         for c in range(math.ceil(w / tile_size)):
16             x0, y0 = c * tile_size, r * tile_size
17             x1 = min(x0 + tile_size, w)
18             y1 = min(y0 + tile_size, h)
19             xs = max(0, x0 - halo); ys = max(0, y0 - halo)
20             xe = min(w, x1 + halo); ye = min(h, y1 + halo)
21             tiles.append(TileInfo(
22                 tile_id=tid, row=r, col=c,
23                 x_start=x0, y_start=y0, x_end=x1, y_end=y1,
24                 left_halo=x0-xs, top_halo=y0-ys,
25                 right_halo=xe-x1, bottom_halo=ye-y1,
26                 data=image[ys:ye, xs:xe],
27             ))
28             tid += 1
29     return tiles

```

Listing 2: Tile extraction with halo borders (src/tiling.py).

3.4 Feature Extraction

Seven features capture per-tile visual complexity:

```

1 def extract_features(tile_bgr: np.ndarray,
2                     row: int, col: int) -> dict:
3     gray = cv2.cvtColor(tile_bgr,
4                          cv2.COLOR_BGR2GRAY).astype(np.float32)
5
6     # Edge density: fraction of Canny edge pixels
7     edges = cv2.Canny(gray.astype(np.uint8), CANNY_LOW,
8                       CANNY_HIGH)

```

```

7     edge_density = float(edges.mean() / 255.0)
8
9     # Gradient variance: Sobel magnitude variance
10    sx = cv2.Sobel(gray, cv2.CV_32F, 1, 0, ksize=3)
11    sy = cv2.Sobel(gray, cv2.CV_32F, 0, 1, ksize=3)
12    gradient_variance = float(np.sqrt(sx**2 + sy**2).var())
13
14    # Intensity variance and histogram entropy
15    intensity_variance = float(gray.var())
16    hist, _ = np.histogram(gray, bins=16, range=(0, 256))
17    histogram_entropy = float(scipy_entropy(hist / hist.sum()
18    + 1e-10))
19
20    # LBP texture score (uniform, P=8, R=1)
21    lbp = local_binary_pattern(gray.astype(np.uint8),
22    P=8, R=1, method="uniform")
23
24    lbp_texture_score = float(lbp.mean())
25
26    return {
27        "edge_density": edge_density,
28        "gradient_variance": gradient_variance,
29        "intensity_variance": intensity_variance,
30        "histogram_entropy": histogram_entropy,
31        "lbp_texture_score": lbp_texture_score,
32        "tile_row": float(row),
33        "tile_col": float(col),
34    }

```

Listing 3: Per-tile feature extraction (src/features.py).

3.5 Complexity Predictor

Eight tabular regressors plus three CNN regressors are trained on the profiled dataset; the best by Spearman ρ is selected:

```

1  TABULAR_MODELS = {
2      "linear": LinearRegression(),
3      "ridge": Ridge(alpha=1.0),
4      "svr": Pipeline([("scaler",
5      StandardScaler()),
6      ("model", SVR(kernel="rbf",
7      C=10))]),
8      "random_forest": RandomForestRegressor(
9      n_estimators=100, n_jobs=-1,
10     random_state=42),
11     "gradient_boosting": GradientBoostingRegressor(
12     n_estimators=100,
13     random_state=42),
14     "xgboost": xgb.XGBRegressor(n_estimators=100,
15     lr=0.1),
16     "lightgbm": lgb.LGBMRegressor(n_estimators=100,

```

```

    lr=0.1),
12     "mlp": Pipeline([("scaler",
13                        StandardScaler()),
14                        ("model", MLPRegressor(
15                           hidden_layer_sizes=(64,
16                                                  32))))]),
17 }
18
19 def fit_tabular(self, X, y):
20     best_rho = -1.0
21     for name, model in TABULAR_MODELS.items():
22         m = deepcopy(model).fit(X, y)
23         rho = spearmanr(y, m.predict(X)).correlation
24         if rho > best_rho:
25             best_rho = rho
26             self.best_tabular_model = name
27             self._best_model = m

```

Listing 4: Tabular model registry (src/predictor.py).

3.6 Scheduler Variants

```

1 class StaticScheduler:
2     def assign(self, tiles, n_workers):
3         # Round-robin tile assignment
4         batches = [[] for _ in range(n_workers)]
5         for i, tile in enumerate(tiles):
6             batches[i % n_workers].append(tile)
7         return batches
8
9 class DynamicScheduler:
10     def get_queue(self, tiles):
11         # FIFO queue; workers pull tasks as they finish
12         return sorted(tiles, key=lambda t: t.tile_id)
13
14 class PredictiveScheduler:
15     def get_queue(self, tiles, predictions):
16         # LPT: longest predicted tile dispatched first
17         order = np.argsort(predictions)[::-1]
18         return [tiles[i] for i in order]
19
20 class HybridScheduler:
21     def get_initial_queue(self, tiles, predictions):
22         # Predictive initial ordering
23         for i, t in enumerate(tiles):
24             self._predicted_times[t.tile_id] =
25                 float(predictions[i])
26         return [tiles[i] for i in
27                 np.argsort(predictions)[::-1]]

```

```

27     def record_completion(self, tile_id, predicted_ms,
28                           actual_ms):
29         # Track prediction errors for online adaptation
        self._errors.append(abs(actual_ms - predicted_ms))

```

Listing 5: Four scheduler classes (src/scheduler.py).

3.7 CPU Processing Pipelines

```

1  def process_tile_cpu_pipeline_a(tile_bgr):
2      # Edge+gradient pipeline (~7 ms/tile). High complexity
       variance.
3      gray = cv2.cvtColor(tile_bgr, cv2.COLOR_BGR2GRAY)
4      blurred = cv2.GaussianBlur(gray, (5, 5), 1.0)
5      sx = cv2.Sobel(blurred, cv2.CV_32F, 1, 0, ksize=3)
6      sy = cv2.Sobel(blurred, cv2.CV_32F, 0, 1, ksize=3)
7      mag = cv2.normalize(cv2.magnitude(sx, sy),
8                          None, 0, 255, cv2.NORM_MINMAX,
9                          cv2.CV_8U)
10     combined = cv2.addWeighted(
11         mag, 0.5, cv2.Canny(blurred, CANNY_LOW, CANNY_HIGH),
12         0.5, 0)
13     return cv2.morphologyEx(combined, cv2.MORPH_CLOSE,
14                             np.ones((MORPH_KERNEL_A,)*2, np.uint8))
15
16 def process_tile_cpu_pipeline_b(tile_bgr):
17     # Denoise+threshold pipeline (~317 ms/tile). Primary
       benchmark.
18     gray = cv2.cvtColor(tile_bgr, cv2.COLOR_BGR2GRAY)
19     bl = cv2.bilateralFilter(
20         gray, BILATERAL_D, BILATERAL_SIGMA, BILATERAL_SIGMA)
21     dn = cv2.fastNlMeansDenoising(bl, None, h=NLM_H)
22     thr = cv2.adaptiveThreshold(dn, 255,
23                                cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11,
24                                2)
25     k = np.ones((MORPH_KERNEL_B,)*2, np.uint8)
26     return cv2.dilate(cv2.morphologyEx(thr, cv2.MORPH_OPEN, k),
27                       k, iterations=1)

```

Listing 6: CPU pipelines A and B (src/worker_cpu.py).

4 Predictor Evaluation (Prototype Results)

4.1 Model Comparison

Table 7: Predictor MAE and Spearman ρ (5-fold CV, GroupShuffleSplit). Best in **bold**.

Model	Raw		Preprocessed	
	MAE	ρ	MAE	ρ
Linear Regression	0.0588	0.543	0.0165	0.534
Ridge	0.0593	0.571	0.0165	0.535
SVR (RBF)	0.0724	0.352	0.0189	—
Random Forest	0.0572	0.542	0.0153	0.584
Gradient Boosting	0.0593	0.533	0.0166	0.507
XGBoost	0.0605	0.452	0.0164	0.520
LightGBM	0.0598	0.482	0.0159	0.538
MLP	0.0787	0.418	0.0533	0.212
SqueezeNet	0.2481	-0.097	—	—
MobileNetV3-Small	0.1385	0.203	—	—
EfficientNet-B0	0.0876	0.003	—	—

5 Scheduling & Parallelism Prototype

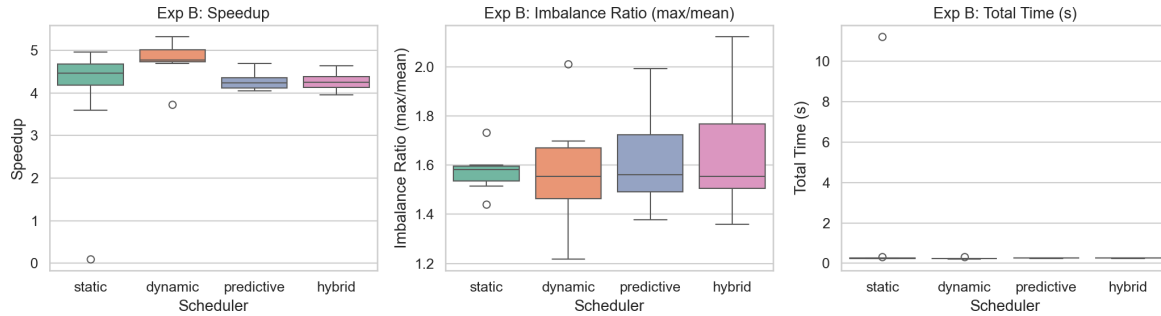


Figure 12: Scheduler comparison at $W = 6$ workers (Pipeline B, 8 Kodak images). Dynamic achieves highest mean speedup ($4.76\times$).

Table 8: Scheduler summary ($W = 6$, Pipeline B).

Scheduler	Speedup	Efficiency	Imbalance Ratio
Dynamic	4.76	0.79	1.57
Static	4.49	0.75	1.60
Predictive	4.28	0.71	1.69
Hybrid	4.27	0.71	1.73

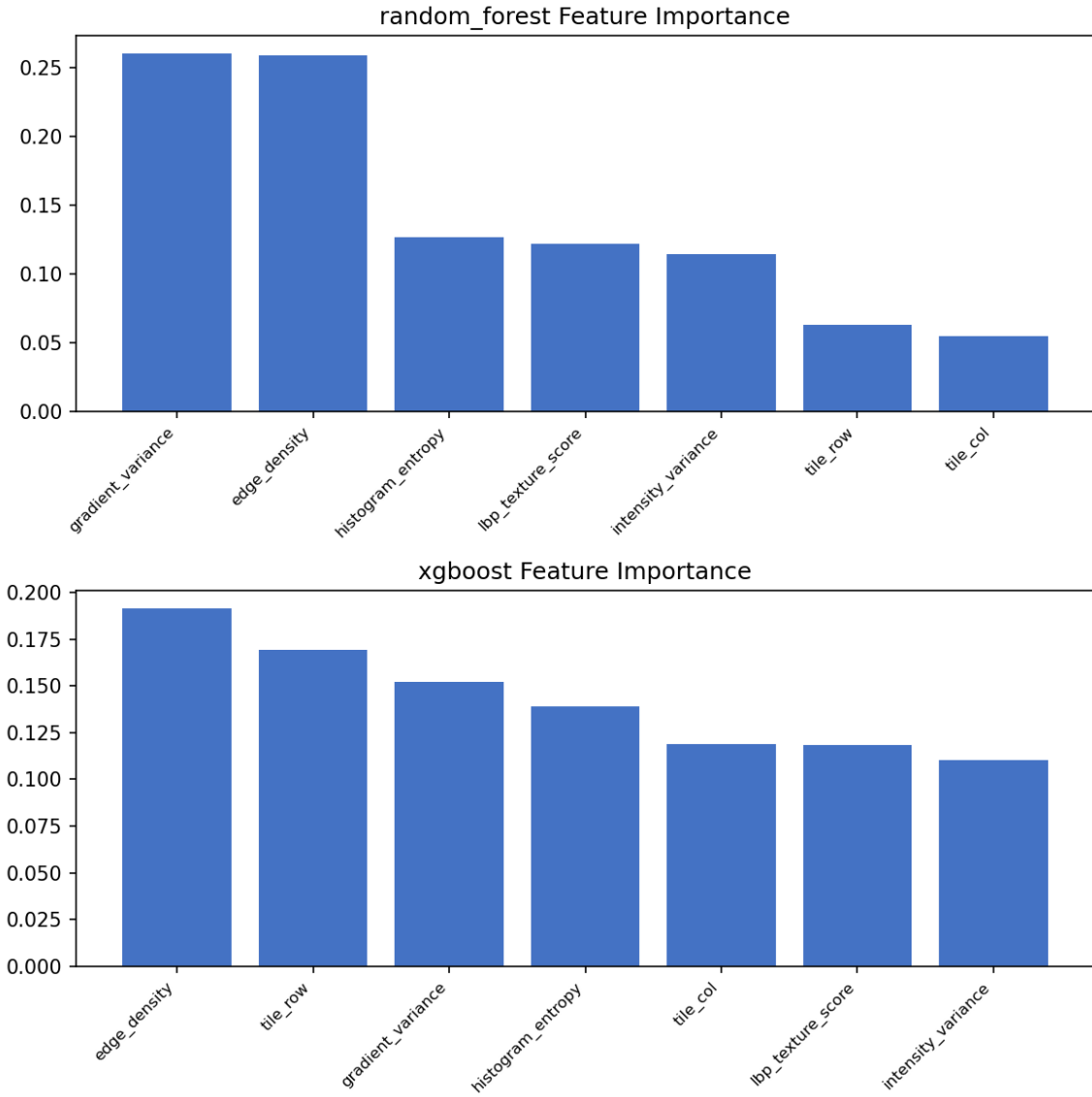


Figure 9: Random Forest feature importances (mean decrease in impurity). Edge density and histogram entropy are the strongest predictors.

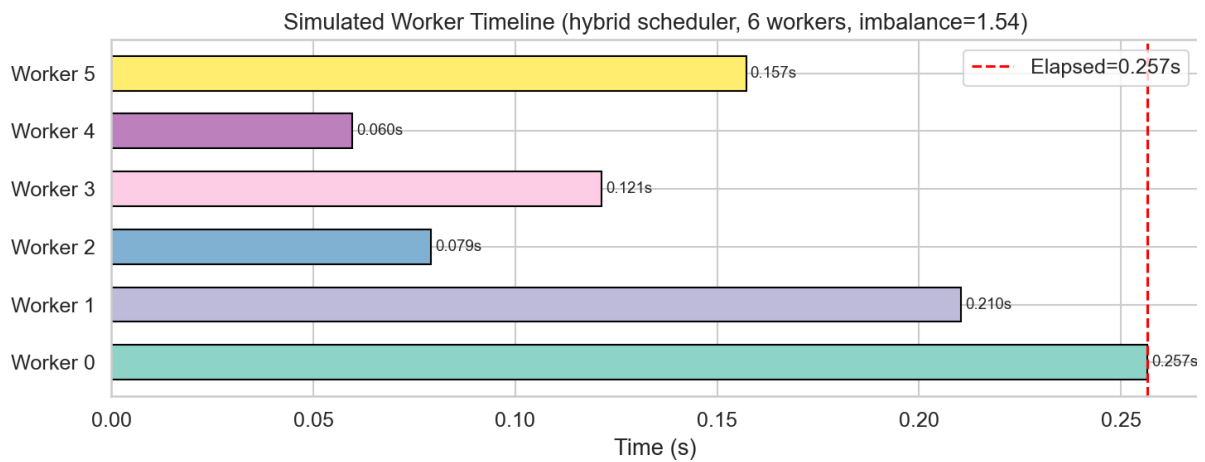


Figure 13: Gantt chart: 6-worker dynamic scheduling on Pipeline B (kodim03). Workers active > 85% of makespan; IR = 1.47.

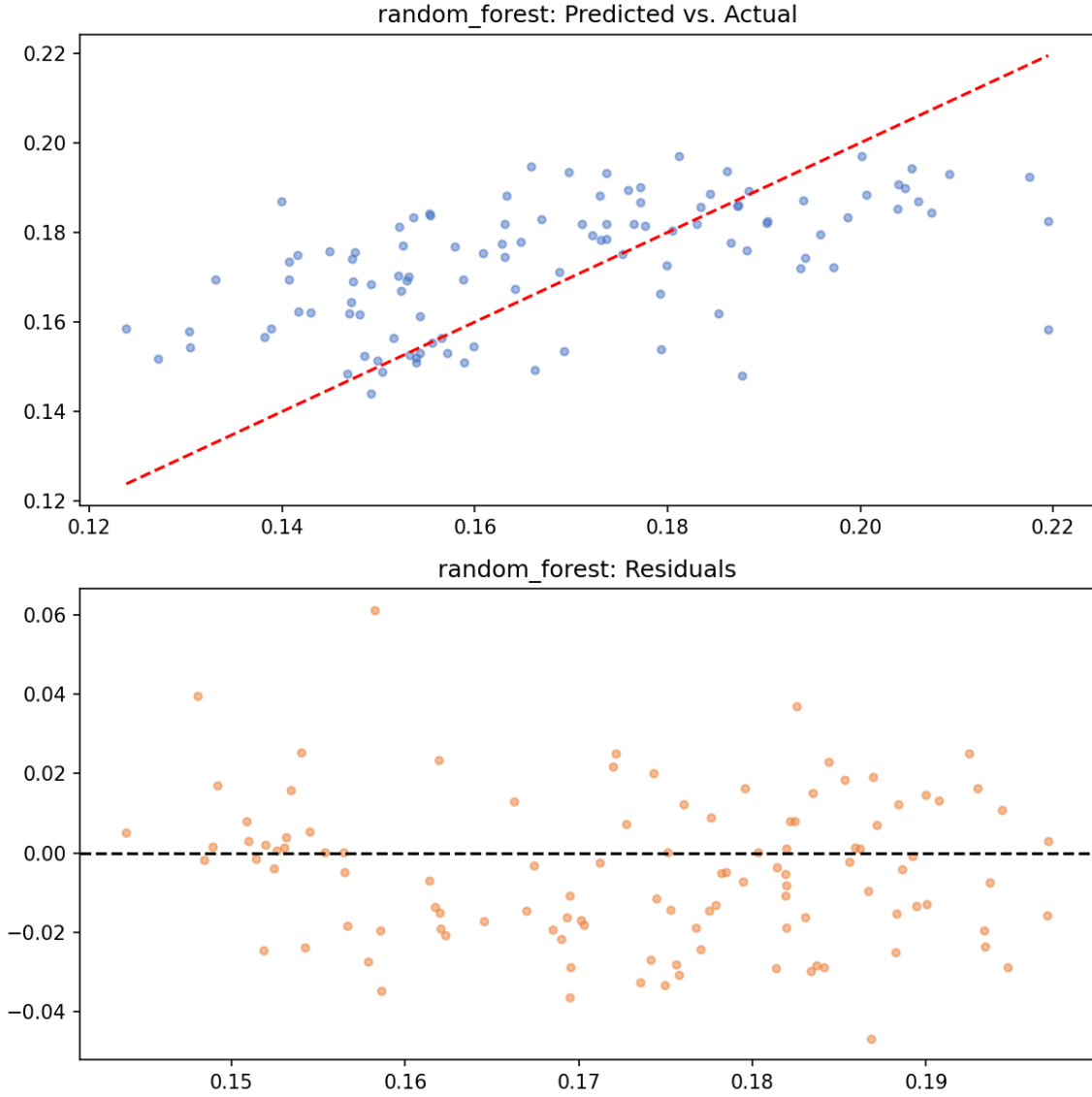


Figure 10: Predicted vs. actual runtime (Random Forest, preprocessed, hold-out fold). Spearman $\rho = 0.584$, $R^2 = 0.254$.

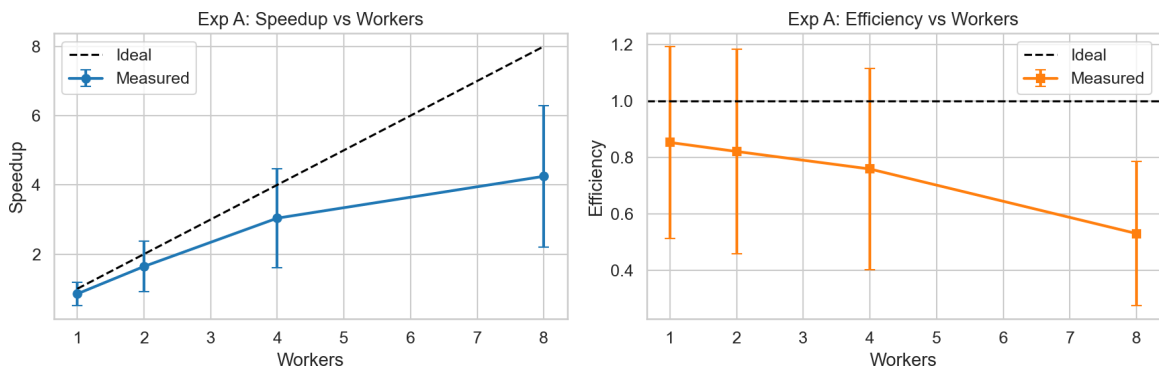
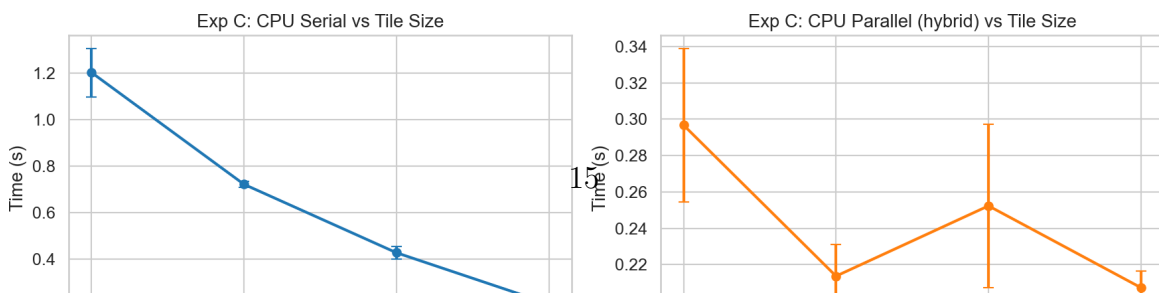


Figure 11: Speedup and parallel efficiency vs. worker count (Pipeline B, dynamic scheduler). Near-linear scaling to $W = 4$ ($3.61\times$, $E = 0.90$). At $W = 8$, efficiency drops to 0.64.



6 Limitations and Future Directions

6.1 Current Limitations

1. **Predictor accuracy** ($R^2 \approx 0.25$): Only 25% of runtime variance is explained. Spearman $\rho \geq 0.75$ is estimated to be needed before predictive scheduling consistently outperforms dynamic.
2. **Lightweight pipeline overhead**: Pipeline A (≈ 7 ms/tile) falls below the multiprocessing spawn overhead, making parallelism counterproductive (speedup < 1).
3. **GPU quality loss**: Kornia GPU pipeline yields SSIM = 0.766 vs. CPU reference; CPU parallel is bit-exact (SSIM = 1.000).
4. **Limited GPU stream benefit**: With only 24 tiles, 4 CUDA streams provide marginal improvement over GPU serial execution.

6.2 Future Work

1. Richer content features: spectral energy, saliency maps, hardware performance counters.
2. Online bandit-based scheduler selection adapting at runtime.
3. Video extension exploiting inter-frame temporal redundancy.
4. Multi-node distributed tiling over InfiniBand interconnects.
5. XAI (SHAP, LIME) to interpret predictor decisions and guide feature engineering.

Conclusion

This report presents the EDA and prototype of AdaptiveTile. Key findings:

- Per-tile runtime is non-normally distributed ($CV = 26.55\%$), validating content-aware complexity prediction as a meaningful problem.
- Preprocessing (Yeo-Johnson + RobustScaler) consistently improves model MAE across all tabular regressors.
- Random Forest achieves best predictor accuracy (Spearman $\rho = 0.584$, MAE = 0.0153 ms), outperforming all CNN models.
- Dynamic work-queue scheduling achieves the highest parallel speedup ($4.76\times$ at $W = 6$).
- GPU serial execution achieves $6.9\times$ mean speedup over CPU serial.
- Tile size $s = 128$ px is optimal; $s \geq 512$ px eliminates parallelism.

These results form the foundation for the final submission, where richer features, higher-accuracy predictors, and extended multi-dataset GPU evaluations will be incorporated.